PORTABLE WORKSPACE

Technical Reference Guide

Encrypted • Containerised • Portable Developer Infrastructure

`Repository: github.com/pnkz01/portable-workspace`

Stack: Shell (67.9%) • Dockerfile (32.1%) • Debian Linux • Docker • KasmVNC

1. Project Overview

The portable-workspace repository is a public infrastructure-as-code template that delivers a security-first, fully reproducible developer workstation on top of Debian Linux. It can run on a cloud VPS, a rented bare-metal host, or a USB-bootable machine — and the entire environment can be rebuilt identically on any Debian host in under an hour.

The project is built around a defence-in-depth philosophy:

- The host OS is hardened: minimal packages, no default users, no password-based SSH
- The disk is fully encrypted at rest using LUKS2 — data is unreadable without the passphrase even if hardware is seized
- All workloads run inside a Docker container — isolated, reproducible, and disposable
- Setup and startup concerns are cleanly separated, making the environment easy to iterate on

> *This architecture is well-suited to DevSecOps environments, security research labs, and remote-first teams that require reproducibility and operational security.*

1.1 Technology Stack

Component	Share / Version	Purpose
Shell Scripts	67.9%	OS init, FDE, SSH hardening, container entrypoints
Dockerfile	32.1%	Container image definitions for shell and GUI workspaces
Docker Compose	—	Service orchestration, volume management, port mapping
LUKS2 / cryptsetup	—	Full disk encryption for host storage
KasmVNC	—	Browser-accessible GUI desktop for the desktop variant
Systemd	—	PID 1 inside containers, enabling normal service management

1.2 Prerequisites

Before beginning setup, ensure the following are available:

- A fresh Debian 12 or Debian 13 installation
- KVM or serial console access (essential for entering the FDE passphrase on every reboot)
- An SSH key pair generated locally: ssh-keygen -t ed25519

- Basic familiarity with the Linux shell and a terminal editor (nano or vim)

2. Architecture & File Structure

The repository is organised into a layered bootstrap sequence. Each layer builds on the previous one: OS hardening first, encryption second, SSH security third, and then the containerised workspace on top.

File	Responsibility
`os_init.sh`	Layer 1 — Prepares the Debian host OS: updates packages, installs cryptsetup, configures GRUB for pre-boot unlock, regenerates initramfs
`fde_rescue.sh`	Layer 2 — Performs in-place full disk encryption from rescue mode. Encrypts root partition, updates crypttab, fstab, GRUB, and initramfs
`hard_sh_ssh.sh`	Layer 3 — Hardens the SSH daemon: regenerates host keys, enforces modern crypto algorithms, creates a new sudo user, disables root and password login
`Dockerfile`	Layer 4 — Defines the Debian 13 Slim shell container with build tools, a non-root user, and systemd as PID 1
`compose.yaml`	Layer 4 — Docker Compose orchestration: persistent volume, port 443, privileged mode, interactive TTY
`cont_setup.sh`	Layer 4 — One-time container provisioning hook (empty by default, customise with dev tools)
`cont_startup.sh`	Layer 4 — Container entrypoint: launches /lib/systemd/systemd as PID 1
`desktop/`	Optional Layer — Full GUI desktop variant using XFCE4, KasmVNC, Firefox, and VS Code

3. Layer-by-Layer Technical Reference

Layer 1: Host OS Initialisation — os_init.sh

Run immediately after provisioning a fresh Debian server. This script prepares the OS for encryption and SSH hardening.

```
nano os_init.sh
chmod +x os_init.sh
sudo ./os_init.sh
sudo reboot now    # reboot into rescue mode
```

What the script does:

- Runs apt update && apt upgrade to bring the system fully up to date
- Installs cryptsetup and cryptsetup-initramfs for full disk encryption support
- Installs essentials tools for development and monitoring utilities
- Disables LLMNR in systemd-resolved to reduce the network attack surface
- Adds GRUB_ENABLE_CRYPTODISK=y to /etc/default/grub to enable pre-boot LUKS unlock
- Runs update-initramfs -u to embed the cryptsetup unlock hooks into the initramfs image
- Prints a system resource summary (IP, open ports, block devices, memory, processes) for verification

> *Note: The chmod +x step is mandatory — freshly cloned scripts do not have the executable bit set. The script must also run with sudo as it modifies system-level configuration.*

Layer 2: Full Disk Encryption — fde_rescue.sh

This is the most security-critical layer. Full disk encryption (FDE) ensures that even if the physical drive is removed from the server, its contents are completely unreadable without the passphrase. Setup must run from rescue mode because the live root partition cannot be encrypted while mounted.

Identifying Disks

Before running the script, identify the drive layout:

```
lsblk       # lists block devices in a tree
fdisk -l    # detailed partition tables
blkid       # UUIDs and filesystem types
```

Running Encryption

```
chmod +x fde_rescue.sh
./fde_rescue.sh -d sda -r sda1 -e sda15
```

Flag	Meaning
`-d <drive>`	The whole disk device — e.g. sda
`-r <root>`	The root partition to encrypt — e.g. sda1
`-e <efi>`	The EFI/boot partition (stays unencrypted for GRUB) — e.g. sda15

What the Script Does Internally

1. Validates that all specified block devices exist before proceeding
2. Runs e2fsck -f to check filesystem integrity
3. Shrinks the ext4 filesystem (by ~32 MB) to create space for the LUKS header
4. Encrypts the partition in-place using cryptsetup reencrypt --encrypt
5. Opens the encrypted device and expands the filesystem back to fill remaining space
6. Converts the PBKDF from Argon2id to PBKDF2 (required for older GRUB versions to unlock the disk)
7. Chroots into the encrypted volume and updates /etc/crypttab, /etc/fstab, and GRUB cmdline
8. Runs grub-install, update-initramfs, and update-grub inside the chroot
9. Unmounts cleanly and closes the LUKS device

> *Cloud hosts: After reboot the system will prompt for the passphrase twice — once by GRUB, once by initramfs. On cloud VPS providers, feed the passphrase through the KVM/serial console. If GRUB does not detect the LUKS drive, boot entries in /etc/grub.d/ may need manual correction.*

Layer 3: SSH Hardening — hard_sh_ssh.sh

With the disk encrypted, SSH is the primary remaining attack surface. This script automates security best practices that are easy to misconfigure manually.

```
chmod +x hard_sh_ssh.sh
sudo ./hard_sh_ssh.sh -o debian -n dubian
```

Flag	Meaning
`-o <old_user>`	The existing username to replace — typically 'debian' from cloud-init
`-n <new_user>`	The new username to create — e.g. 'dubian'

Cryptographic Hardening

The script regenerates all SSH host keys and drops everything except:

- Host key algorithms: Ed25519 and RSA-4096 only
- Key exchange algorithms: sntrup761x25519-sha512, curve25519-sha256, diffie-hellman-group16/18-sha512
- Ciphers: chacha20-poly1305, aes256-gcm, aes256-ctr, aes128-gcm, aes128-ctr
- MACs: hmac-sha2-256-etm, hmac-sha2-512-etm, umac-128-etm
- Moduli: entries below 3071 bits are filtered out from /etc/ssh/moduli

sshd_config Changes

```
Port 1111                          # move off default port 22
PermitRootLogin no                 # disable root login
PasswordAuthentication no          # key-only authentication
PubkeyAuthentication yes
RequiredRSASize 3072               # minimum RSA key size
```

User Migration

- Creates new sudo user with the specified name
- Copies ~/.ssh/authorized_keys from the old user to the new user
- Sets /bin/bash as the default shell
- Grants passwordless sudo via /etc/sudoers.d/

Dynamic MOTD

A system metrics script is installed at /etc/update-motd.d/10-uname and displayed on every SSH login, showing: current date/time, CPU and memory usage, disk usage, uptime, top 5 processes by CPU, and currently active users.

> *Critical: Always open a second SSH session as the new user to verify it works before deleting the old user. Locking yourself out of a remote server with no console access is not recoverable without physical or KVM access.*

Layer 4: Container Workspace — Dockerfile & compose.yaml

The container provides a clean, isolated, and reproducible workspace that is independent of the host OS configuration. It runs Debian 13 Slim with systemd as PID 1.

Base Image & User

```
FROM debian:13-slim

ENV USER=dubian \
    SUDO_GROUP=sudo

RUN groupadd -r dubian && useradd -m -g dubian dubian \
    && usermod -aG sudo dubian \
    && sed -i '/^%sudo/s/ALL$/NOPASSWD:ALL/g' /etc/sudoers
```

Installed Packages

- sudo, systemd, systemd-sysv — init system and privilege management
- build-essential, libffi-dev, libssl-dev — compilation toolchain
- python3-dev, git, nano, curl, wget — development utilities
- procps, ca-certificates, iproute2 — system and network utilities

Entrypoint

The container entrypoint launches systemd directly as PID 1 rather than a simple shell. This enables proper service management — including VNC in the desktop variant — inside the container.

```
# cont_startup.sh
exec /lib/systemd/systemd
```

Docker Compose Configuration

```
services:
  container_bass:
    build: .
    hostname: dubiandubaian
    volumes:
      - dubian_data:/home/dubian
    ports:
      - "443:443"
    privileged: true
    stdin_open: true
    tty: true

volumes:
  dubian_data:
```

Setting	Purpose
`privileged: true`	Required for systemd as PID 1 and any low-level container operations
`stdin_open + tty`	Equivalent to docker run -it — keeps the container interactive
`dubian_data volume`	Persists the user home directory across container rebuilds
`port 443`	Exposes HTTPS — can be used for local dev server or tunnelling

Launching the Container

```
# Build and start
sudo docker compose up -d

# Attach a shell as the workspace user
sudo docker compose exec container_bass bash -c "su dubian -P -c 'bash'"

# Inside the container
ls -alh
```

4. GUI Desktop Variant — desktop/

The desktop/ subdirectory provides a complete graphical workspace accessible from any browser. It adds a full XFCE4 desktop, KasmVNC (a browser-native VNC server), Firefox, and VS Code on top of the base Debian image.

4.1 Additional Software

- XFCE4 + xfce4-goodies — lightweight, full-featured desktop environment
- KasmVNC v1.4.0 — browser-based VNC server (HTTPS, no client install required)
- Firefox — installed from Mozilla's official signed apt repository
- Visual Studio Code — installed from Microsoft's official signed apt repository

4.2 VNC Configuration

The cont_setup.sh script for the desktop variant configures VNC at build time:

- Creates VNC credentials in ~/.kasmpasswd using vncpasswd
- Registers a systemd user service (kasmvnc@:1.service) that starts XFCE4 via vncserver
- Generates a self-signed TLS certificate (RSA-4096, 10-year validity) for HTTPS
- Installs the certificate to /etc/ssl/certs/ and key to /etc/ssl/private/

> *Default VNC credentials are set in desktop/cont_setup.sh: user, password. Change these before building the image for any non-local deployment.*

4.3 Docker Compose (Desktop)

```
services:
  vnc:
    build: .
    hostname: debaian
    volumes:
      - debaian_data:/home/debaian
    ports:
      - "8444:8444"
    privileged: true
    stdin_open: true
    tty: true
```

4.4 Accessing the Desktop

```
cd desktop
sudo docker compose up -d

# Visit in your browser:
# https://<server-ip>:8444
```

```
# Debug VNC issues:
sudo docker compose exec vnc bash -c "su debaian -P -c 'bash'"
tail -f ~/.vnc/*.log
```

5. End-to-End Setup Walkthrough

The following is the complete setup sequence from a fresh Debian server to a running workspace container.

Step 1 — Clone & Initialise the Host

```
git clone https://github.com/pnkz01/portable-workspace.git
cd portable-workspace
chmod +x os_init.sh
sudo ./os_init.sh
sudo reboot now
```

Step 2 — Encrypt the Disk (from rescue mode)

```
lsblk && fdisk -l && blkid
chmod +x fde_rescue.sh
./fde_rescue.sh -d sda -r sda1 -e sda15
# Reboot and unlock via KVM/serial console
```

Step 3 — Harden SSH

```
chmod +x hard_sh_ssh.sh
sudo ./hard_sh_ssh.sh -o debian -n dubian
# Open a SECOND SSH session as 'dubian' and verify before continuing
sudo userdel -r debian
sudo visudo /etc/sudoers.d/90-cloud-init-users   # remove stale entries
sudo nano ~/.ssh/authorized_keys                  # add/rotate your key
```

Step 4 — Install Docker CE

```
sudo apt update && sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
    -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

Step 5 — Launch the Workspace

```
# Recommended: Docker Compose
sudo docker compose up -d
sudo docker compose exec container_bass bash -c "su dubian -P -c 'bash'"

# Alternative: Plain Docker
sudo docker build -t container_bass .
sudo docker run --privileged -it -d --name container_bass container_bass
sudo docker exec -it container_bass /bin/bash -c "su dubian -P -c 'bash'"
```

Step 6 (Optional) — Launch GUI Desktop

```
cd desktop
# Edit cont_setup.sh and change VNC_PW from 'changeme' first!
sudo docker compose up -d
# Visit https://<server-ip>:8444 in your browser
```

6. Security Design & Rationale

The project applies a layered defence-in-depth model. Each security measure addresses a distinct threat.

Control	Threat Addressed	Rationale
LUKS2 Full Disk Encryption	Physical hardware seizure	Data at rest is unreadable without the passphrase regardless of how the disk is accessed
PBKDF2 key derivation	GRUB compatibility	Older GRUB versions do not support Argon2id; PBKDF2 enables pre-boot unlock via the bootloader
SSH key-only authentication	Remote brute-force attacks	Eliminates password guessing entirely — only holders of the private key can connect
Non-default SSH port (1111)	Automated scan noise	Moves off port 22 to reduce volume of automated attack traffic in logs
Ed25519 + RSA-4096 host keys	Host spoofing / MITM	Stronger key types replace weaker DSA/ECDSA defaults
Moduli filtering (>=3071 bits)	Logjam / weak DH attacks	Removes Diffie-Hellman parameters below safe threshold from the moduli database
Modern cipher/MAC suite only	Protocol downgrade attacks	Disables legacy algorithms; only post-2020 recommended algorithms are permitted
Non-root container user	Container privilege escalation	Running as 'dubian' with NOPASSWD sudo is safer than running workloads as root
Systemd as container PID 1	Zombie process accumulation	Proper init system reaps orphan processes and manages service lifecycle correctly
Named Docker volumes	Data loss on rebuild	Home directory persists across docker compose down and image rebuilds

7. Customisation Guide

7.1 Customising the Shell Container

The cont_setup.sh file in the root directory is intentionally empty and is the designated place to add container-level tooling. It runs once at docker build time, so packages installed here are baked into the image.

Examples of what to add:

- Language runtimes: Node.js (via nvm), Python packages (pip), Go, Rust
- Shell configuration: clone dotfiles, set up .bashrc / .zshrc / .vimrc
- Git globals: user.name, user.email, default branch name

7.2 Customising the Desktop Container

Before building the desktop image:

- Edit desktop/cont_setup.sh and change VNC_PW from the default 'changeme' to a strong password
- The VNC_USER variable sets the username for browser login (default: 'cries')
- The self-signed TLS certificate subject can be customised in the openssl req command

7.3 Resetting Docker

To fully reset the Docker environment (removes all containers, images, and volumes):

```
sudo docker stop $(sudo docker ps -a -q)
sudo docker rm $(sudo docker ps -a -q)
sudo docker volume rm $(sudo docker volume ls -q)
sudo docker rmi $(sudo docker images -a -q)
sudo docker system prune -a -f --volumes
```

> *Warning: The volume rm command will delete the dubian_data and debaian_data volumes, permanently removing any data stored in the container home directories.*

8. Troubleshooting Reference

Symptom	Likely Cause	Resolution
GRUB does not detect LUKS drive after reboot	PBKDF mismatch or missing grub.cfg entry	Run cryptsetup luksConvertKey --pbkdf pbkdf2 on the partition and re-run update-grub inside the chroot
System asks for passphrase more than twice	Expected behaviour	GRUB requests it once, then initramfs requests it again
Locked out of SSH after running hard_sh_ssh.sh	Old session closed before verifying new user	Use KVM/serial console to access the host. Always verify new user in a second shell before logging out
Container exits immediately on docker compose up	systemd not running as PID 1, or missing --privileged	Ensure the compose.yaml has privileged: true and that cont_startup.sh executes systemd
VNC page unreachable at port 8444	KasmVNC service not started, or TLS certificate issue	Exec into the container and run: tail -f ~/.vnc/*.log to inspect VNC startup errors
VNC login fails with correct credentials	Credentials not set before build	Edit VNC_PW in desktop/cont_setup.sh, then rebuild with docker compose build --no-cache
fde_rescue.sh fails with 'wrong drive ids'	Incorrect -d / -r / -e arguments	Re-run lsblk and blkid to confirm partition names, then rerun the script with correct flags
e2fsck fails at start of fde_rescue.sh	Partition is mounted	The script must run from rescue mode — not from the live running OS

9. Quick Reference

Key File Locations

Path	Purpose
`/etc/ssh/sshd_config`	Main SSH server configuration
`/etc/ssh/sshd_config.d/hardening.conf`	Crypto algorithm restrictions set by hard_sh_ssh.sh
`/etc/default/grub`	GRUB settings — includes GRUB_ENABLE_CRYPTODISK=y
`/etc/crypttab`	LUKS device unlock configuration for boot
`/etc/fstab`	Filesystem mount table — updated to reference /dev/mapper/root_crypt
`/etc/update-motd.d/10-uname`	Dynamic MOTD script showing system metrics on SSH login
`/home/dubian/.kasmpasswd`	VNC credentials file (desktop variant)
`/etc/ssl/certs/ssl-cert-snakeoil.pem`	TLS certificate for KasmVNC HTTPS (desktop variant)

Useful Commands

```
# Check LUKS status
cryptsetup status root_crypt
cryptsetup luksDump /dev/sda1

# Check SSH daemon status
sudo systemctl status sshd
sudo sshd -T | grep -E 'port|permitrootlogin|passwordauth'

# Container operations
sudo docker compose ps
sudo docker compose logs -f
sudo docker compose restart

# Check open ports
ss -tulpn
```